



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

N° de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 05 de Septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Ejercicio 1. Iniciales URC de diferentes colores

Descripción

El primer ejercicio consistió en dibujar las iniciales U, R y C, utilizando un color diferente para cada letra:

- U: rojo.
- R: verde.
- C: azul.

Para realizar las letras se reutilizaron los cuadrados previamente creados y se modificaron mediante transformaciones geométricas. Cada parte de las letras se obtuvo cambiando principalmente la posición y la escala de una instancia del cuadrado.

Por ejemplo, para formar la letra U se utilizaron dos barras verticales y una barra horizontal inferior. Para la R se utilizaron barras verticales, horizontales y una parte diagonal. Finalmente, la C se construyó con una barra vertical y dos barras horizontales.

```

339 //Letra U - Roja
340
341 //Lado izquierdo
342 DibujarFigura(
343     2,
344     -1.05f, 0.0f, -4.0f,
345     0.10f, 1.0f,
346     0.0f,
347     uniformModel
348 );
349
350 //Lado derecho
351 DibujarFigura(
352     2,
353     -0.65f, 0.0f, -4.0f,
354     0.10f, 1.0f,
355     0.0f,
356     uniformModel
357 );
358
359 //Parte inferior
360 DibujarFigura(
361     2,
362     -0.85f, -0.45f, -4.0f,
363     0.50f, 0.10f,
364     0.0f,
365     uniformModel
366 );

```

Figura 1. Instancias utilizadas para formar la letra U de color rojo.

```

370 //Letra R - Verde
371
372 //Lado izquierdo
373 DibujarFigura(
374     3,
375     -0.25f, 0.0f, -4.0f,
376     0.10f, 1.0f,
377     0.0f,
378     uniformModel
379 );
380
381 //Parte superior
382 DibujarFigura(
383     3,
384     -0.05f, 0.45f, -4.0f,
385     0.50f, 0.10f,
386     0.0f,
387     uniformModel
388 );
389
390 //Parte central
391 DibujarFigura(
392     3,
393     -0.05f, 0.05f, -4.0f,
394     0.50f, 0.10f,
395     0.0f,
396     uniformModel
397 );
398
399 //Lado derecho superior
400 DibujarFigura(
401     3,
402     0.20f, 0.25f, -4.0f,
403     0.10f, 0.50f,
404     0.0f,
405     uniformModel
406 );
407
408 //Parte diagonal
409 DibujarFigura(
410     3,
411     0.08f, -0.23f, -4.0f,
412     0.10f, 0.62f,
413     25.0f,
414     uniformModel
415 );

```

Figura 2. Transformaciones utilizadas para formar la letra R de color verde.

```

419 //Letra C - Azul
420
421 //Parte superior
422 DibujarFigura(
423     8,
424     0.85f, 0.45f, -4.0f,
425     0.55f, 0.10f,
426     0.0f,
427     uniformModel
428 );
429
430 //Lado izquierdo
431 DibujarFigura(
432     8,
433     0.60f, 0.0f, -4.0f,
434     0.10f, 1.0f,
435     0.0f,
436     uniformModel
437 );
438
439 //Parte inferior
440 DibujarFigura(
441     8,
442     0.85f, -0.45f, -4.0f,
443     0.55f, 0.10f,
444     0.0f,
445     uniformModel
446 );

```

Figura 3. Instancias utilizadas para formar la letra C de color azul.

Resultado

Al ejecutar el programa se mostraron las tres iniciales al mismo tiempo. Cada una mantiene un color diferente y está formada a partir de varias instancias de la misma figura, modificando su tamaño y posición.

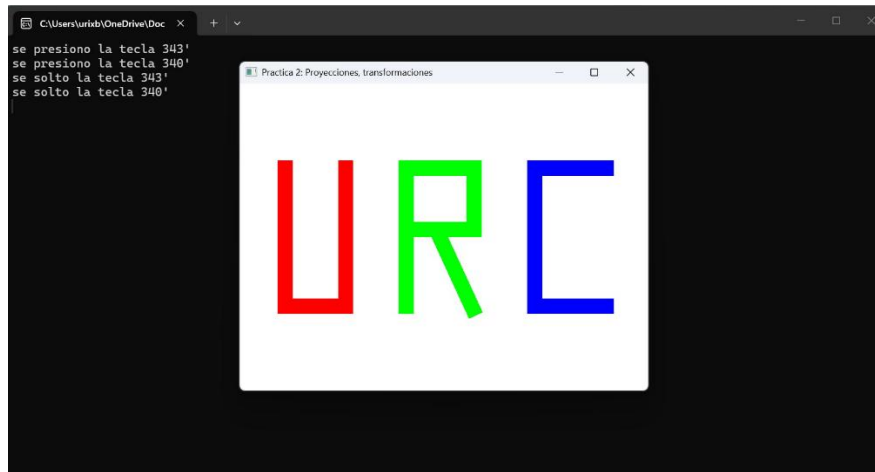


Figura 4. Resultado final de las iniciales URC con diferentes colores.

Ejercicio 2. Dibujo de las 3 figuras a partir de instanciación de pirámides y cubos

Descripción

En el segundo ejercicio se retomaron las tres composiciones realizadas anteriormente, pero ahora se sustituyeron las figuras planas por objetos tridimensionales. El programa ya contaba con una pirámide creada mediante cuatro vértices y un cubo formado por ocho vértices. Ambos objetos se almacenan dentro de meshList para poder reutilizarlos varias veces.

```

58 //Pirámide triangular regular
59 void CreaPiramide()
60 {
61     unsigned int indices[] = {
62         0,1,2,
63         1,3,2,
64         3,0,2,
65         1,0,3
66     };
67     GLfloat vertices[] = {
68         -0.5f, -0.5f, 0.0f, //0
69         0.5f, -0.5f, 0.0f, //1
70         0.0f, 0.5f, -0.25f, //2
71         0.0f, -0.5f, -0.5f, //3
72     };
73     Mesh *obj1 = new Mesh();
74     obj1->CreateMesh(vertices, indices, 12, 12);
75     meshList.push_back(obj1);
76 }

```

```

80 //Vértices de un cubo
81 void CrearCubo()
82 {
83     unsigned int cubo_indices[] = {
84         // front
85         0, 1, 2,
86         2, 3, 0,
87         // right
88         1, 5, 6,
89         6, 2, 1,
90         // back
91         7, 6, 5,
92         5, 4, 7,
93         // left
94         4, 0, 3,
95         3, 7, 4,
96         // bottom
97         4, 5, 1,
98         1, 0, 4,
99         // top
00         3, 2, 6,
01         6, 7, 3
02     };
03
04     GLfloat cubo_vertices[] = {
05         // front
06         -0.5f, -0.5f, 0.5f,
07         0.5f, -0.5f, 0.5f,
08         0.5f, 0.5f, 0.5f,
09         -0.5f, 0.5f, 0.5f,
10         // back
11         -0.5f, -0.5f, -0.5f,
12         0.5f, -0.5f, -0.5f,
13         0.5f, 0.5f, -0.5f,
14         -0.5f, 0.5f, -0.5f
15     };
16     Mesh *cubo = new Mesh();
17     cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
18     meshList.push_back(cubo);
19 }

```

Figura 5. Vértices e índices utilizados para crear la pirámide y el cubo.

Creación de los shaders

Para poder utilizar los mismos objetos con diferentes colores se crearon archivos shader distintos.

Se utilizaron shaders para:

- Rojo.
- Verde.
- Azul.
- Café.
- Magenta.

- Amarillo.
- Negro.

Las rutas de estos archivos se declararon en el programa y después cada shader se cargó dentro de shaderList.

```

19 //Dimensiones de la ventana
20 const float toRadians = 3.14159265f/180.0; //grados a radianes
21 Window mainWindow;
22 std::vector<Mesh*> meshList;
23 std::vector<MeshColor*> meshColorList;
24 std::vector<Shader*> shaderList;
25 //Vertex Shader
26 static const char* vShader = "shaders/shader.vert";
27 static const char* fShader = "shaders/shader.frag";
28 static const char* vShaderColor = "shaders/shadercolor.vert";
29 static const char* fShaderColor = "shaders/shadercolor.frag";
30 //shaders nuevos se crearian acá
31 static const char* vShaderRojo = "shaders/shaderrojo.vert";
32 static const char* vShaderVerde = "shaders/shaderverde.vert";
33 static const char* vShaderAzul = "shaders/shaderazul.vert";
34 static const char* vShaderCafe = "shaders/shadercafe.vert";
35 static const char* vShaderMagenta = "shaders/shadermagenta.vert";
36 static const char* vShaderAmarillo =
37 "shaders/shaderamarillo.vert";
38
39 static const char* vShaderNegro =
40 "shaders/shadernegro.vert";
41
42
43 float angulo = 0.0f;
44
45 //color café en RGB : 0.478, 0.255, 0.067
46
47 const int PIRAMIDE = 0;
48 const int CUBO = 1;
49
50 const int ROJO = 2;
51 const int VERDE = 3;
52 const int AZUL = 4;
53 const int CAFE = 5;
54 const int MAGENTA = 6;
55 const int AMARILLO = 7;
56 const int NEGRO = 8;

```

```

275 void CreateShaders()
276 {
277     //Shader original para Mesh
278     Shader* shader1 = new Shader();
279     shader1->CreateFromFiles(vShader, fShader);
280     shaderList.push_back(*shader1);
281
282     //Shader para MeshColor
283     Shader* shader2 = new Shader();
284     shader2->CreateFromFiles(vShaderColor, fShaderColor);
285     shaderList.push_back(*shader2);
286
287
288     //Shader rojo
289     Shader* shaderRojo = new Shader();
290     shaderRojo->CreateFromFiles(vShaderRojo, fShaderColor);
291     shaderList.push_back(*shaderRojo);
292
293
294     //Shader verde
295     Shader* shaderVerde = new Shader();
296     shaderVerde->CreateFromFiles(vShaderVerde, fShaderColor);
297     shaderList.push_back(*shaderVerde);
298
299
300     //Shader azul
301     Shader* shaderAzul = new Shader();
302     shaderAzul->CreateFromFiles(vShaderAzul, fShaderColor);
303     shaderList.push_back(*shaderAzul);
304
305
306     //Shader cafe
307     Shader* shaderCafe = new Shader();
308     shaderCafe->CreateFromFiles(vShaderCafe, fShaderColor);
309     shaderList.push_back(*shaderCafe);
310
311
312     //Shader magenta
313     Shader* shaderMagenta = new Shader();
314     shaderMagenta->CreateFromFiles(vShaderMagenta, fShaderColor);
315     shaderList.push_back(*shaderMagenta);

```

Figura 6. shaders utilizados para modificar el color de los objetos.

Transformaciones de los objetos 3D

Para reutilizar las pirámides y cubos se utilizó la función DibujarObjeto().

Esta función recibe datos como:

- El objeto que se quiere dibujar.
- El shader correspondiente.
- La posición en X, Y y Z.
- La escala en los tres ejes.
- Las rotaciones en X, Y y Z.
- La matriz de proyección.

Dentro de esta función se aplican glm::translate, glm::rotate y glm::scale.

```

336 void DibujarObjeto(
337     int objeto,
338     int shaderIndice,
339     float x, float y, float z,
340     float escalaX, float escalaY, float escalaZ,
341     float rotX, float rotY, float rotZ,
342     glm::mat4 projection)
343 {
344     //Usar shader correspondiente
345     shaderList[shaderIndice].useShader();
346
347     GLuint uniformModel =
348         shaderList[shaderIndice].getModelLocation();
349
350     GLuint uniformProjection =
351         shaderList[shaderIndice].getProjectLocation();
352
353     //Matriz modelo
354     glm::mat4 model = glm::mat4(1.0f);
355
356     //Posicion
357     model = glm::translate(
358         model,
359         glm::vec3(x, y, z)
360     );
361
362     //Rotacion en X
363     model = glm::rotate(
364         model,
365         glm::radians(rotX),
366         glm::vec3(1.0f, 0.0f, 0.0f)
367     );
368
369     //Rotacion en Y
370     model = glm::rotate(
371         model,
372         glm::radians(rotY),
373         glm::vec3(0.0f, 1.0f, 0.0f)
374     );
375

```

```

377 //Rotacion en Z
378 model = glm::rotate(
379     model,
380     glm::radians(rotZ),
381     glm::vec3(0.0f, 0.0f, 1.0f)
382 );
383
384 //Escala
385 model = glm::scale(
386     model,
387     glm::vec3(
388         escalaX,
389         escalaY,
390         escalaZ
391     )
392 );
393
394
395 glUniformMatrix4fv(
396     uniformModel,
397     1,
398     GL_FALSE,
399     glm::value_ptr(model)
400 );
401
402 glUniformMatrix4fv(
403     uniformProjection,
404     1,
405     GL_FALSE,
406     glm::value_ptr(projection)
407 );
408
409 meshList[objeto]->RenderMesh();
410 }

```

Figura 7. Función utilizada para aplicar transformaciones a las pirámides y cubos.

Esta función permitió usar varias veces el mismo cubo o la misma pirámide sin tener que crear nuevamente sus vértices.

Como ahora se trabajó con objetos tridimensionales, se activó la prueba de profundidad mediante:

```
glEnable(GL_DEPTH_TEST);
```

También se limpió el buffer de profundidad en cada ciclo:

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

Esto permite que OpenGL determine correctamente qué superficies deben aparecer delante o detrás de otras.

Generación de las tres composiciones

En la primera figura se utilizaron cubos cafés alargados para sustituir las barras laterales y pirámides de diferentes colores para sustituir los triángulos.

En la figura central se utilizaron varias pirámides alrededor de un cubo azul y un cubo café. Para mejorar la apariencia tridimensional se modificaron las escalas en los tres ejes y se realizaron rotaciones en X, Y y Z.

En la tercera figura se utilizaron cuatro pirámides con distintos colores y orientaciones.

El código utiliza llamadas repetidas a DibujarObjeto() cambiando únicamente el objeto, shader y valores de transformación.

<pre> 457 //Base negra 458 DibujarObjeto(459 CUBO, NEGRO, 460 0.0f, -0.78f, -4.30f, 461 462 2.65f, 0.08f, 0.18f, 463 464 -10.0f, 10.0f, 0.0f, 465 466 projection 467); 468 469 //Figura izquierda 470 471 //Barra cafe izquierda 472 DibujarObjeto(473 CUBO, CAFE, 474 -1.10f, -0.19f, -4.05f, 475 476 0.08f, 1.05f, 0.22f, 477 478 -10.0f, 18.0f, 0.0f, 479 480 projection 481); 482 483 //Barra cafe derecha 484 DibujarObjeto(485 CUBO, CAFE, 486 -0.70f, -0.19f, -4.05f, 487 488 0.08f, 1.05f, 0.22f, 489 490 -10.0f, 18.0f, 0.0f, 491 492 projection 493); 494 495 </pre>	<pre> 511 //Piramide roja 512 DibujarObjeto(513 PIRAMIDE, ROJO, 514 -0.90f, -0.279f, -3.80f, 515 516 0.35f, 0.29f, 0.45f, 517 518 -12.0f, 16.0f, 180.0f, 519 520 projection 521); 522 523 //Piramide verde 524 DibujarObjeto(525 PIRAMIDE, VERDE, 526 -0.90f, -0.573f, -3.80f, 527 528 0.35f, 0.29f, 0.45f, 529 530 -12.0f, 16.0f, 180.0f, 531 532 projection 533); 534 535 </pre>
--	--

Figura 8. Instancias de cubos y pirámides utilizadas en la primera figura.

```

538 //Figura central
539
540 //Piramide amarilla superior izquierda
541 DibujarObjeto(
542     PIRAMIDE, AMARILLO,
543     -0.285f, -0.055f, -4.0f,
544
545     0.54f, 0.27f, 0.42f,
546
547     -10.0f, 14.0f, 45.0f,
548
549     projection
550 );
551
552 //Piramide roja superior derecha
553 DibujarObjeto(
554     PIRAMIDE, ROJO,
555     0.285f, -0.055f, -4.0f,
556
557     0.54f, 0.27f, 0.42f,
558
559     -10.0f, 14.0f, -45.0f,
560
561     projection
562 );
563
564
565 //Piramide magenta inferior izquierda
566 DibujarObjeto(
567     PIRAMIDE, MAGENTA,
568     -0.285f, -0.625f, -4.0f,
569
570     0.54f, 0.27f, 0.42f,
571
572     -10.0f, 14.0f, 135.0f,
573
574     projection
575 );
576
577
578 //Piramide verde inferior derecha
579 DibujarObjeto(
580     PIRAMIDE, VERDE,
581     0.285f, -0.625f, -4.0f,
582
583     0.54f, 0.27f, 0.42f,
584
585     -10.0f, 14.0f, 225.0f,
586
587     projection
588 );
589
590
591 //Cubo azul central
592 DibujarObjeto(
593     CUBO, AZUL,
594     0.0f, -0.34f, -3.58f,
595

```

```

592 //Cubo azul central
593 DibujarObjeto(
594     CUBO, AZUL,
595     0.0f, -0.34f, -3.58f,
596     0.50f, 0.50f, 0.38f,
597     -16.0f, 22.0f, 45.0f,
600     projection
601 );
602
603
604 //Cubo cafe central
605 DibujarObjeto(
606     CUBO, CAFE,
607     0.0f, -0.34f, -3.18f,
608     0.27f, 0.17f, 0.28f,
609     -16.0f, 22.0f, 45.0f,
612     projection
613 );
614
615

```

Figura 9. Transformaciones aplicadas a los objetos de la figura central.

```

619 //Figura derecha
620 |
621 //Piramide magenta superior
622 DibujarObjeto(
623     PIRAMIDE, MAGENTA,
624     0.95f, -0.15f, -4.0f,
625     0.40f, 0.38f, 0.45f,
626     -12.0f, 16.0f, 0.0f,
629     projection
630 );
631
632
633 //Piramide verde inferior izquierda
634 DibujarObjeto(
635     PIRAMIDE, VERDE,
636     0.75f, -0.53f, -4.0f,
637     0.40f, 0.38f, 0.45f,
638     -12.0f, 16.0f, 0.0f,
641     projection
642 );
643
644
645 //Piramide roja inferior derecha
646 DibujarObjeto(
647     PIRAMIDE, ROJO,
648     1.15f, -0.53f, -4.0f,
649     0.40f, 0.38f, 0.45f,
650     -12.0f, 16.0f, 0.0f,
653     projection
654 );
655
656
657

```

```

660 //Piramide amarilla central invertida
661 DibujarObjeto(
662     PIRAMIDE, AMARILLO,
663     0.95f, -0.53f, -3.55f,
664     0.40f, 0.38f, 0.45f,
665     -12.0f, 16.0f, 180.0f,
666     projection
667 );
668
669
670

```

Figura 10. Pirámides utilizadas para formar la tercera figura.

Resultado

Después de realizar varios ajustes de posición, escala y rotación, se logró generar las tres composiciones utilizando pirámides y cubos en lugar de figuras planas.

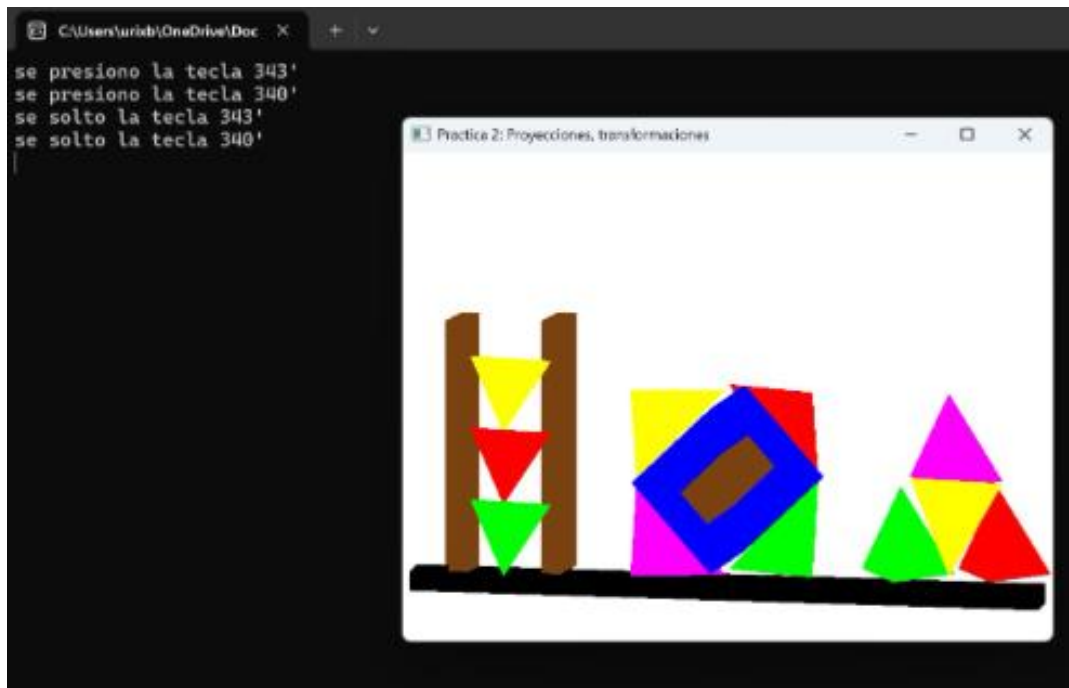


Figura 11. Resultado final de las composiciones utilizando objetos tridimensionales.

Problemas presentados

Durante los dos ejercicios se me presentaron algunos problemas relacionados principalmente con las transformaciones.

En el ejercicio de las iniciales tuve que ajustar varias veces las posiciones y escalas para que las partes de las letras quedaran correctamente alineadas, especialmente en la letra R debido a su parte diagonal.

En el segundo ejercicio, inicialmente las pirámides y cubos se veían muy parecidos a figuras planas. Para corregir esto tuve que activar el buffer de profundidad y utilizar rotaciones en los ejes X, Y y Z.

También se tuvo que modificar varias veces las escalas del cubo azul y del cubo café de la figura central, ya que al principio se veían demasiado pequeños o deformados. Además, fue necesario revisar los shaders para que los colores aparecieran correctamente.

Conclusión:

- a) Estos ejercicios tuvieron una complejidad mayor, principalmente por el uso de varias transformaciones, instancias y shaders dentro de una misma escena. El ejercicio de las iniciales fue más sencillo porque se trabajó con figuras planas, mientras que el de pirámides y cubos requirió más ajustes para que los objetos conservaran una apariencia tridimensional y los colores fueran correctos.
- b) En general realizar los ejercicios de esta práctica ayudaron a comprender mejor la utilidad de reutilizar una misma figura mediante instancias y transformaciones, así como la forma en que los shaders permiten cambiar el color de un objeto sin modificar sus vértices.
- c) Conclusión: Al finalizar esta práctica se concluye que el uso de transformaciones geométricas, instancias y shaders permite reutilizar una misma figura para generar diferentes composiciones sin necesidad de crear cada objeto desde cero. También se comprendió mejor la diferencia entre trabajar con figuras planas y objetos tridimensionales, así como la importancia de ajustar correctamente la posición, escala, rotación y profundidad para obtener el resultado esperado.

Bibliografía

- Universitat Jaume I. (s. f.). *Tema 3: Transformaciones geométricas*. Repositori UJI. <https://repositori.uji.es/server/api/core/bitstreams/4eff1602-1eb0-4d80-a151-97116>
- Mondragon, G. (2026). *OpenGL moderno con C++: Construye objetos 3D paso a paso*. Udemy. <https://www.udemy.com/course/opengl-moderno-con-c-construye-objetos-3d-paso-a-paso/>